Lecture 3: Data Representation II — Addresses, Pointers & Arrays

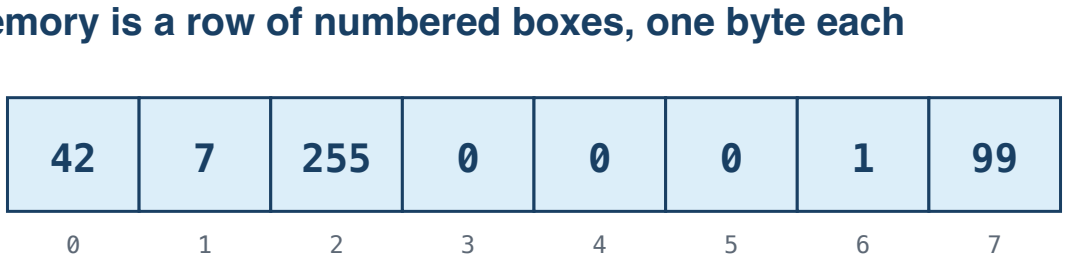
Session 3 · Wed Sep 2, 2026

Contents

Memory Is a Row of Numbered Boxes . . . 1
Pointers Demystified . . . 2
Three Things, Kept Straight . . . 2
Two Pointers, One Object . . . 2
Pointer Arithmetic . . . 3
Run It: the Addresses Step by Four . . . 3
An Array Is Not a Pointer . . . 4
Arrays Decay to Pointers . . . 4
Strings Are Char Arrays . . . 4
Two-Dimensional Arrays . . . 5
Run It: Rows End to End . . . 5
Why the Order Matters . . . 5
The Stack . . . 6
Pointers to Pointers . . . 6
Why `char **argv` . . . 6
A Matrix, Two Ways . . . 7
Null Pointers . . . 7
Practice . . . 8
Reading . . . 8

Memory Is a Row of Numbered Boxes

Last time we filled in one byte. Today: where the bytes *live*.



Every byte your program can touch has a number — its **address**. Nothing more mysterious than house numbers along a street — though they run large, so we write them in hex: `0x7ffd4a08`, not 2147305992.

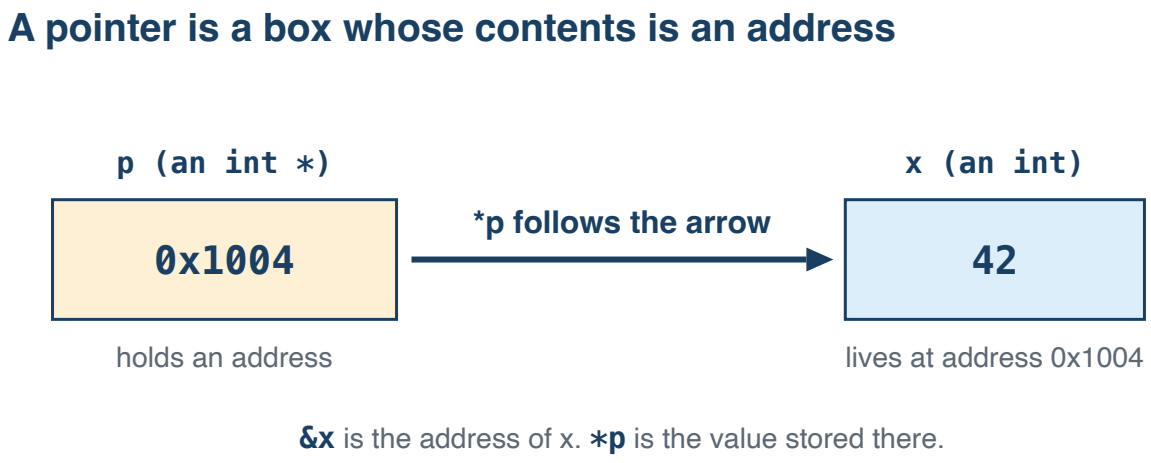
> **Note**
>
> This row is your program's own **virtual address space**. Each running program gets one, and the operating system keeps them apart *(Lecture 9)*. An optimizing compiler may also keep a value only

in a register, where it has no address at all.

Pointers Demystified

A **pointer** is a variable whose value is an address.

```c
int x = 42;
int *p = &x;          // &x is "the address of x"
printf("%d\n", *p);   // 42 — *p is "the value at that address"
```



Three Things, Kept Straight

Expression	Type	Value
`x`	`int`	42
`&x`	`int *`	the address of `x`
`*p`	`int`	42

Read the declaration `int *p` as "`*p` is an `int`" — which is exactly what the last row says.

Tip

`&` and `*` undo each other: `*&x` is `x`.

Two Pointers, One Object

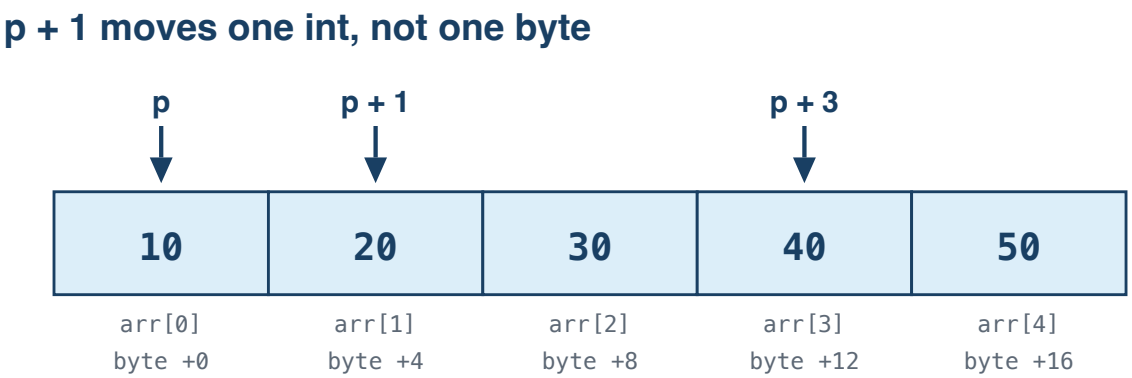
Nothing stops two pointers from designating the same object:

```c
int x = 42;
int *p = &x;
int *q = &x;          // q points at the same int

*p = 99;
printf("%d\n", *q);   // 99 — q sees the change
printf("%d\n", x);    // 99 — and so does x
```

Two names, one box. This is called **aliasing**. It is why a function handed two pointers may not assume they point at different things — and exactly why `memmove` exists alongside `memcpy`, for the case where the two regions overlap.

Pointer Arithmetic




```c
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;        // p points at arr[0]

*(p + 2) = 99;       // arr[2] becomes 99
```

Rule: `p + i` moves `i * sizeof(*p)` bytes — you count elements, the compiler counts bytes. Subtraction matches: `&arr[3] - &arr[0]` is `3`, not `12`. And `a[i]` is *defined* as `*(a + i)`, in the standard itself.

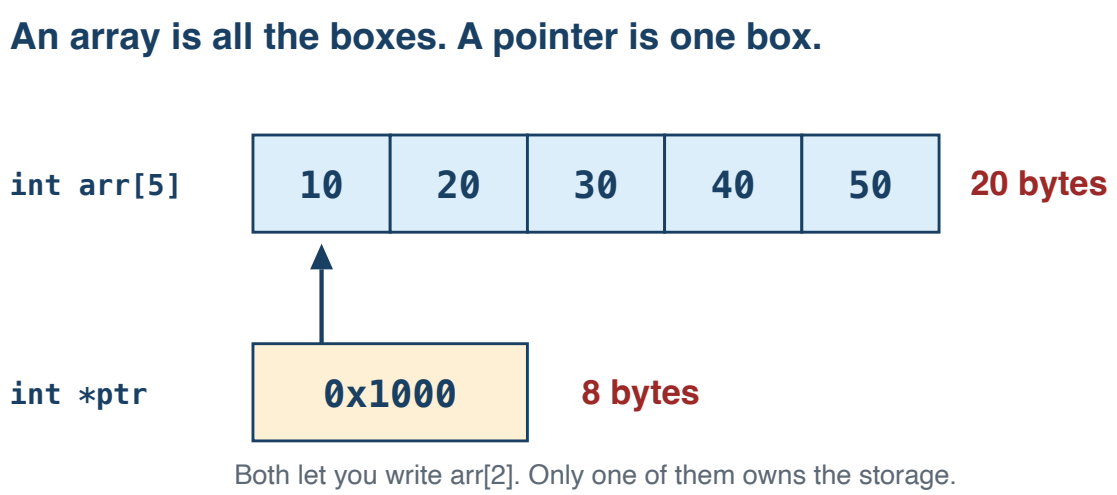
Run It: the Addresses Step by Four

```c
int arr[5] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++)
    printf("arr[%d] = %-3d  &arr[%d] = %p\n", i, arr[i], i, (void *)&arr[i]);
printf("&arr[3] - &arr[0] = %ld\n", (long)(&arr[3] - &arr[0]));
```

```
arr[0] = 10   &arr[0] = 0x16b106960
arr[1] = 20   &arr[1] = 0x16b106964
arr[2] = 30   &arr[2] = 0x16b106968
arr[3] = 40   &arr[3] = 0x16b10696c
arr[4] = 50   &arr[4] = 0x16b106970
&arr[3] - &arr[0] = 3
```

`…960, 964, 968, 96c, 970` — four apart, every time. Your addresses will be different numbers; the spacing will not.

An Array Is Not a Pointer




```c
int arr[5];       // five ints of storage
int *ptr = arr;   // one address, pointing into that storage

sizeof(arr);      // 20 on our teaching systems (5 × 4)
sizeof(ptr);      // 8  on our teaching systems

arr = ptr;        // ERROR: an array name is not assignable
```

Arrays Decay to Pointers

In a parameter declaration, C **adjusts** `int a[]` to `int *a`. These two lines declare the same function:

```c
void foo(int a[]);   // what you wrote
void foo(int *a);    // what the compiler sees
```

So the array-ness is gone by the time the function body runs:

```c
void foo(int a[]) {
    sizeof(a);       // 8 here — a is a pointer
}
```

The length does not travel with the pointer. Pass it yourself:

```c
void foo(int *a, size_t n);
```

Strings Are Char Arrays

C has no string type. A "string" is a `char` array ending in a zero byte, `'\0'` — that terminator is how every library function knows where to stop.

```c
#include <string.h>

char        arr[] = "hello";   // 6 bytes: h e l l o \0 — writable
const char *ptr   = "hello";   // a pointer to a string literal

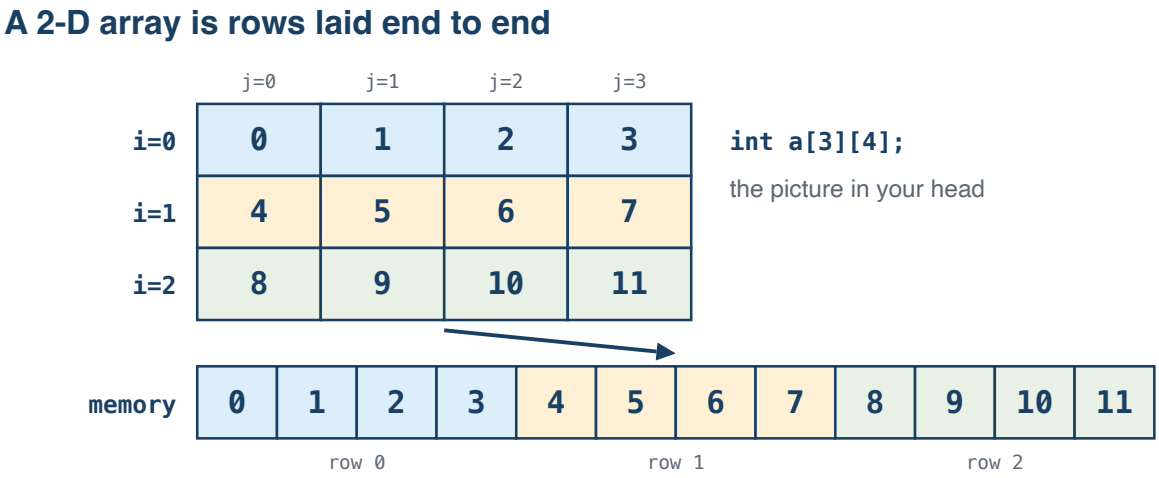
sizeof(arr);   // 6  — the array, terminator included
strlen(arr);   // 5  — characters before the terminator
sizeof(ptr);   // 8  — just a pointer
arr[0] = 'H';  // fine
ptr[0] = 'H';  // won't compile — and that is the point
```

Tip

Say `const char *` for a string literal. Plain `char *` compiles too, but then writing through it is undefined behavior your compiler never mentions. `const` turns a crash you might not hit into an error you cannot miss.

Two-Dimensional Arrays

`int a[3][4]` is not a grid in memory. It is three rows of four, stored one row after the next — **row-major** order.




```c
int a[3][4];
sizeof(a);       // 48 — the whole thing
sizeof(a[0]);    // 16 — one row of four ints
```

Run It: Rows End to End

```c
int a[3][4];
for (int i = 0; i < 3; i++, printf("  <- row %d\n", i - 1))
    for (int j = 0; j < 4; j++) printf("%p ", (void *)&a[i][j]);
```

```
0x16fac6948 0x16fac694c 0x16fac6950 0x16fac6954   <- row 0
0x16fac6958 0x16fac695c 0x16fac6960 0x16fac6964   <- row 1
0x16fac6968 0x16fac696c 0x16fac6970 0x16fac6974   <- row 2
```

Read across, then down: `…954` ends row 0 and `…958` starts row 1 — the next four bytes. There is no gap between the rows, because there are no rows. It is one run of twelve ints.

Why the Order Matters

Walking a 2-D array along rows touches neighbouring bytes. Walking it down columns jumps a whole row every step:

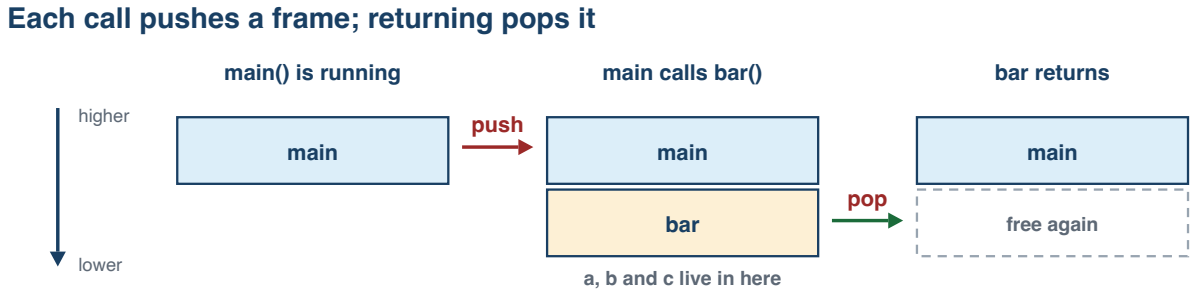
```c
for (i..) for (j..) sum += a[i][j];   // row-major: steps 4 bytes
for (j..) for (i..) sum += a[i][j];   // column-major: jumps 4*ncols bytes
```

Same additions, same answer. In Lecture 1 the column-major version took **26 times longer** on a 4096×4096 array. Now you can see why the two loops are not the same journey through memory — Lecture 15 explains what the hardware does about it.

The Stack

Lecture 1 told you returning `&local` is broken and you took it on faith. This is where those locals live, and why the pointer outlives them.

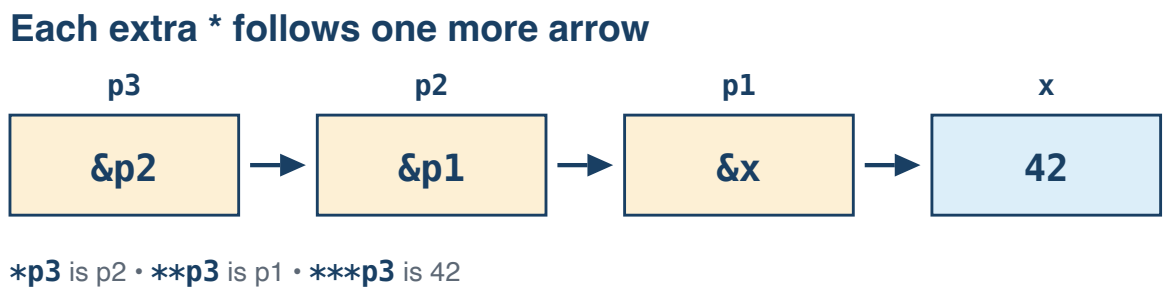
```
void bar(void) { int a = 1, b = 2, c = 3; }
```



The frame is real; where `a`, `b` and `c` sit *inside* it is not specified — your machine's layout is stable, and that is the trap, not the reassurance.

Pointers to Pointers

```
int x = 42;
int   *p1 = &x;    // pointer to int
int  **p2 = &p1;   // pointer to pointer to int
int ***p3 = &p2;   // and one more
```

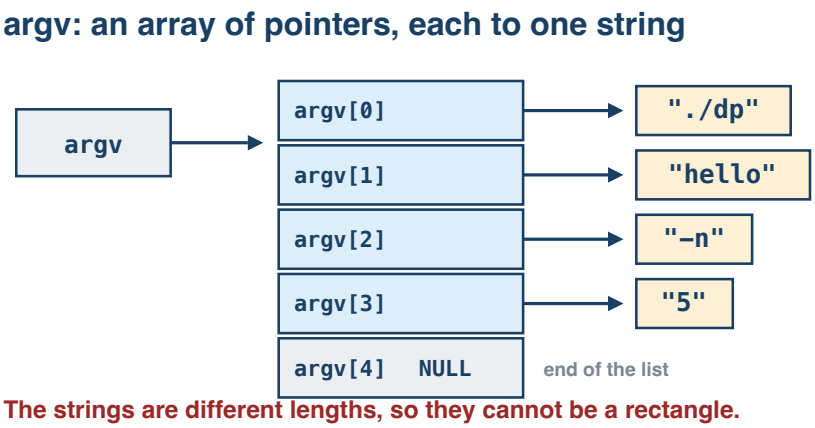


You have already used one: `main(int argc, char **argv)` receives a sequence of pointers, each pointing at one command-line argument.

Why `char **argv`

Every program you write already receives one:

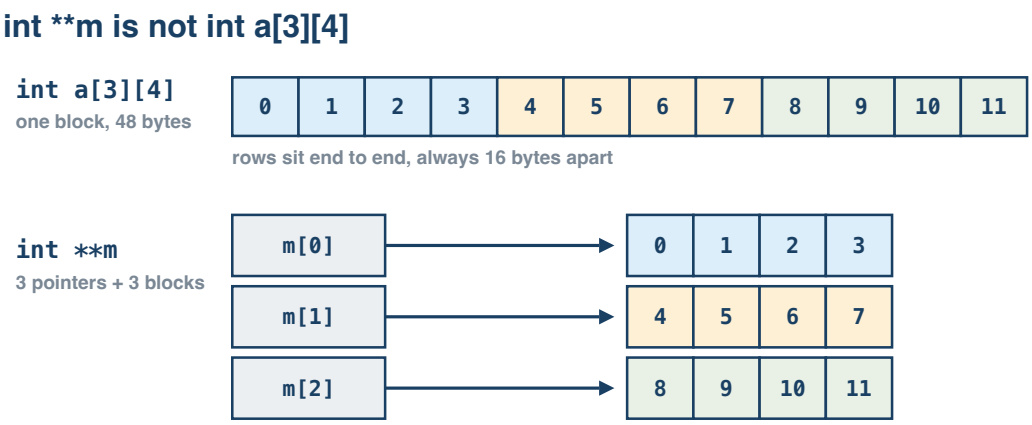
```
for (int i = 0; i < argc; i++)
    printf("argv[%d] = \"%s\"\n", i, argv[i]);
```



`./prog hello -n 5` gives `argc == 4`, and `argv[argc]` is `NULL`.

A Matrix, Two Ways

```c
int **m = malloc(rows * sizeof *m);
for (int i = 0; i < rows; i++)
    m[i] = malloc(cols * sizeof *m[i]);   // each row on its own
```



Both are written `m[i][j]`, and that is where the resemblance ends. `sizeof(a)` is 48 and `&a[0][0] + 6` is `&a[1][2]`; `sizeof(m)` is 8, and walking off the end of one row is undefined even when the addresses happen to line up.

Null Pointers

`NULL` is the pointer value that means "points at nothing".

```c
int *p = NULL;
*p = 42;          // undefined behavior — on our systems, usually a crash
```

Check before you follow:

```c
if (p != NULL) {
    *p = 42;
}
```

Note

"Usually a crash" is the *good* case: the failure is loud and immediate. Undefined behavior is not required to crash at all.

Practice

1. Given `int a[10]; int *p = a;` — why does `p + 1` refer to `a[1]` rather than to the next byte?
2. `p` is an `int *` holding `0x1000`. What address is in `p + 2`? What if `p` had been a `char *` instead?
3. Why does `sizeof` report 40 inside `main` but 8 inside a function you passed the array to?
4. `char *s = "hello"; s[0] = 'H';` compiles. What happens when you run it, and why does `char s[] = "hello"; s[0] = 'H';` behave differently?
5. Write a function taking `int **` that swaps which array two pointers refer to.

Reading

- CS:APP §3.8 (Array Allocation and Access)
- The Descent to C — a careful pointer explainer